

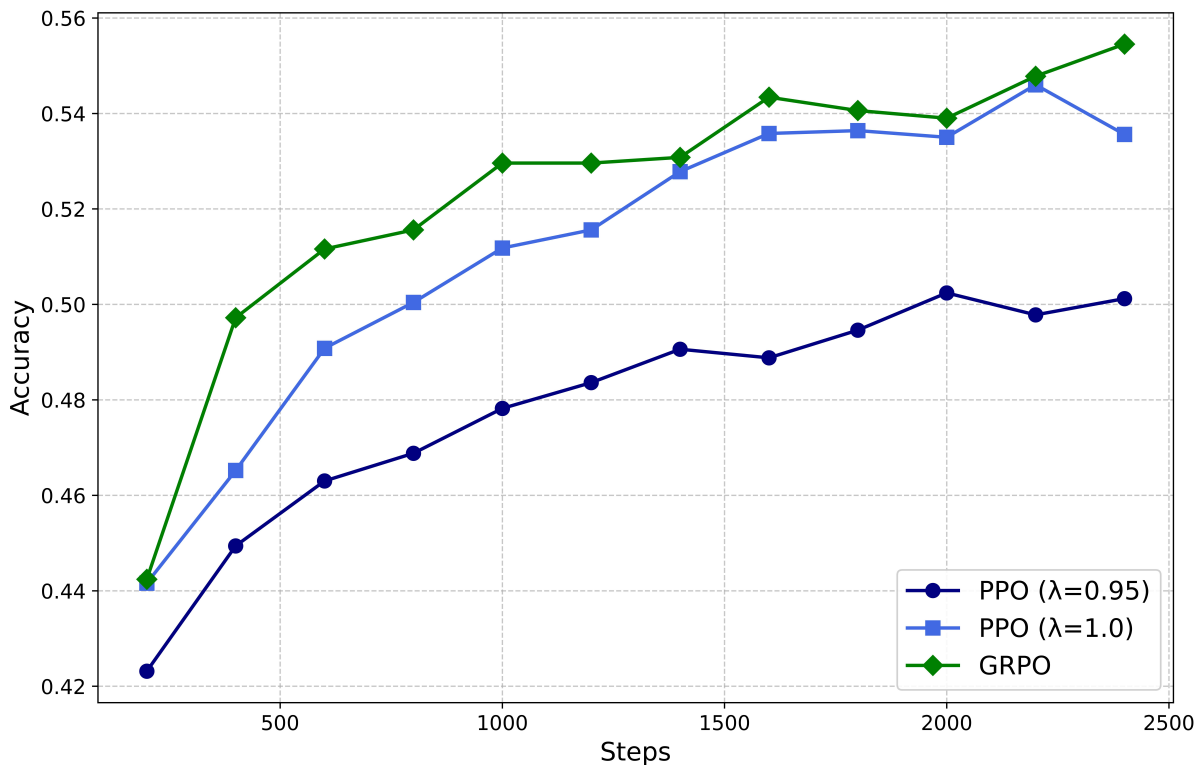


Figure 4 | Performance of PPO and GRPO on the MATH task.

maximize cumulative rewards, PPO’s approach penalizes the cumulative KL divergence, which may implicitly penalize the length of the response and thereby prevent the model’s response length from increasing. In addition, as we may train thousands of steps in the scenario of training long chain-of-thought reasoning models, the trained policy can diverge significantly from the initial reference policy. In order to balance the scope that the training policy can explore and the stability of the training, we periodically update the reference policy to the latest policy during the actual training process.

Figure 4 compares the performance of PPO and GRPO on the MATH task using DeepSeek-Coder-V2-Lite (16B MoE with 2.4B active parameters). Unlike GRPO, PPO requires additional hyperparameter tuning—particularly of the λ coefficient in GAE—and is highly sensitive to this parameter. When λ is set to 0.95 (the default value in most open-source PPO implementations), PPO performs considerably worse than GRPO. However, with careful tuning (setting λ to 1.0), PPO’s performance improves substantially, nearing that of GRPO.

While PPO can achieve comparable performance when appropriately tuned, it demands additional computational cost for hyperparameter optimization. Moreover, considering the memory and computational overhead associated with training an additional value model, GRPO presents a more practical alternative, especially when training large-scale models with constrained resources.

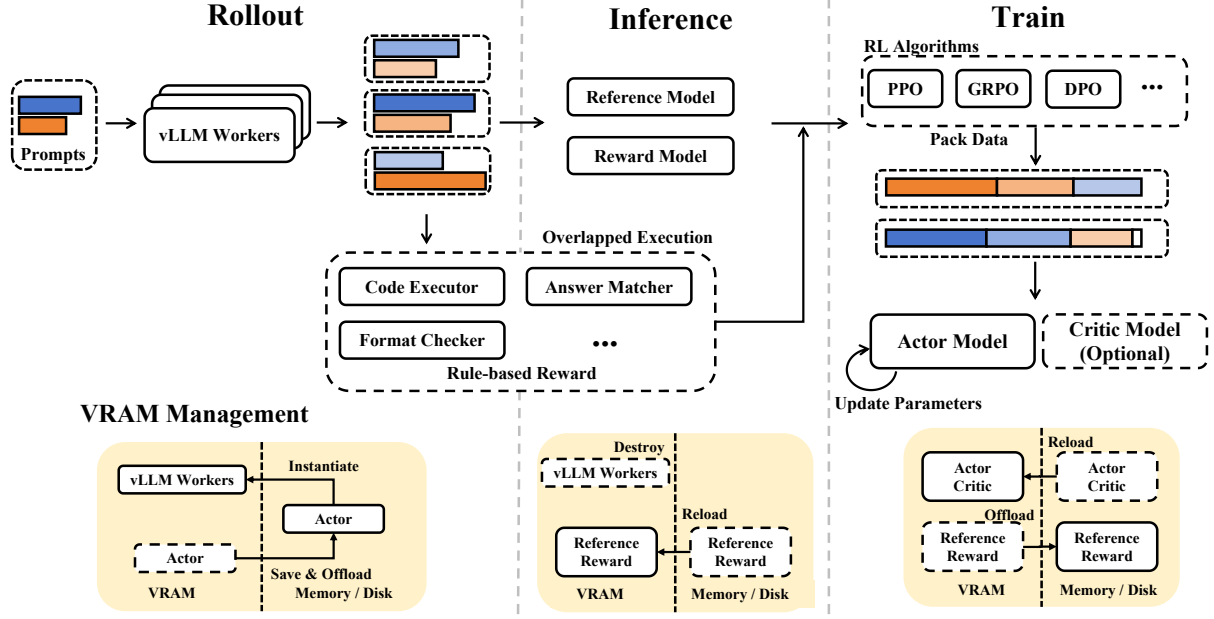


Figure 5 | Overview of our RL framework.

B. Training Details

B.1. RL Infrastructure

Conducting RL training on large models places high demands on the infrastructure. Our RL framework is architected with a decoupled and extensible structure to facilitate seamless integration of diverse models and algorithms. Within this framework, we have incorporated both intra-modular and inter-modular optimization techniques, to ensure training efficiency and scalability.

Specifically, as depicted in Figure 5, the framework is partitioned into four distinct modules, each corresponding to a specific phase of the RL pipeline:

- **Rollout Module:** Prompts are loaded from training dataset and uniformly dispatched across multiple vLLM (Kwon et al., 2023) workers, each equipped with the actor model, to sample multiple responses. For DeepSeek-V3 MoE architecture, we implement an expert parallelism strategy across nodes to reduce memory access overhead, and deploy redundant copies of hotspot experts to balance computational loads among different experts. Multi-Token Prediction (MTP) component is also leveraged for self-speculative decoding, significantly accelerating the decoding speed and effectively minimizing the completion time for the longest samples.
- **Inference Module:** This module loads the reward model and reference to perform a forward pass on the samples generated during the rollout phase, thereby obtaining model-based rewards and other essential information.
- **Rule-based Reward Module:** This module computes rule-based rewards for the model-generated responses. A unified interface has been designed to accommodate diverse implementations (e.g., code executor, answer matcher, format checker, etc.). Although this module does not require loading models into GPU memory, its execution tends to be time-consuming. To tackle this issue, an asynchronous scheduling approach is employed to overlap its execution with the Rollout and Inference modules, effectively hiding the

associated latency.

- **Training Module:** This module loads the actor model and the critic model (if required), to compute loss and update model parameters. It provides flexible support for a variety of RL algorithms (e.g., PPO, GRPO, DPO, etc.). To minimize computational waste caused by sequence padding and balance the workload across devices, we design the following data packing strategy: first, all data in a global batch is sorted by length and distributed across processes within the data parallel group; subsequently, within each process, the Best-Fit strategy is applied to pack the data into fixed-length chunks with minimal padding; finally, the number of chunks is adjusted to be equal across all processes. Additionally, we have integrated the DualPipe algorithm, utilized in DeepSeek-V3 training, to achieve efficient pipeline parallelism.

Notably, upon completion of each module (excluding the Rule-based Reward module), the model instances utilized in that phase are automatically offloaded from VRAM to either system memory or disk storage, thereby freeing up VRAM for the subsequent phase.

B.2. Reward Model Prompt

Please act as an impartial judge and evaluate the quality of the responses provided by two AI assistants to the user prompt displayed below. You will be given assistant A's answer and assistant B's answer. Your job is to evaluate which assistant's answer is better. Begin your evaluation by generating your own answer to the prompt. You must provide your answers before judging any answers.

When evaluating the assistants' answers, compare both assistants' answers with your answer. You must identify and correct any mistakes or inaccurate information.

Then consider if the assistant's answers are helpful, relevant, and concise. Helpful means the answer correctly responds to the prompt or follows the instructions. Note when user prompt has any ambiguity or more than one interpretation, it is more helpful and appropriate to ask for clarifications or more information from the user than providing an answer based on assumptions. Relevant means all parts of the response closely connect or are appropriate to what is being asked. Concise means the response is clear and not verbose or excessive.

Then consider the creativity and novelty of the assistant's answers when needed. Finally, identify any missing important information in the assistants' answers that would be beneficial to include when responding to the user prompt.

After providing your explanation, you must output only one of the following choices as your final verdict with a label:

1. Assistant A is significantly better: `[[A>>B]]`
 2. Assistant A is slightly better: `[[A>B]]`
 3. Tie, relatively the same: `[[A=B]]`
 4. Assistant B is slightly better: `[[B>A]]`
 5. Assistant B is significantly better: `[[B>>A]]`
- Example output: My final verdict is tie: `[[A=B]]`.